AnyFlow: Any-Step Video Diffusion Model with On-Policy Flow Map Distillation

Yuchao Gu[1,2], *Guian Fang*[2], *Yuxin Jiang*[2], *Weijia Mao*[2], *Song Han*[1,3], Han Cai[1*], Mike Zheng Shou[2*]

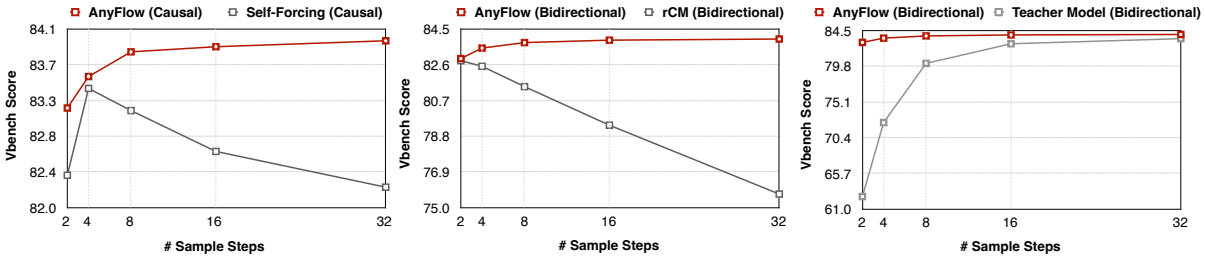
[1] *NVIDIA*
[2] *Show Lab, National University of Singapore*
[3] *MIT*
https://nvlabs.github.io/AnyFlow

1 Introduction

Video diffusion models have recently achieved remarkable generation quality at scale [1–5], but most pipelines remain tied to fixed inference-step budgets. In practice, users require flexible generation: rapid previews for quick iteration and higher-fidelity outputs for final delivery. This motivates the need for *any-step* models that can trade latency for quality at test time without retraining.

Existing few-step video distillation methods are predominantly built on consistency models [6–10]. While effective under very small sampling budgets, they do not provide robust any-step behavior. As illustrated in fig. 1, representative methods (e.g., rCM [9] for bidirectional models and Self-Forcing [10] for causal models) often degrade as the number of sampling steps increases, instead of improving with additional computation. The limitation is structural: consistency formulations emphasize fixed-point mappings to z_0. During multi-step sampling, repeatedly re-noising intermediate states introduces cumulative bias, causing trajectories to drift away from the target PF-ODE path [11] rather than being progressively refined, as shown in fig. 2(a).

To address this gap, we propose AnyFlow, the first any-step video diffusion distillation framework based on a two-time flow map formulation [11–13]. Flow maps generalize endpoint consistency by learning transitions between arbitrary time pairs, i.e., $z_t \to z_r$ rather than only $z_t \to z_0$ (as shown in fig. 2(b)), which naturally supports variable step sizes and inference budgets.



(a) Quantitative comparison of AnyFlow test-time scaling against consistency-based methods and the teacher model.

[width=]8figures/videosrc/teaser/0000000020

(b) Qualitative comparison of AnyFlow test-time scaling against consistency-based methods and the teacher model.

FIGURE 1 Test-Time Scaling of AnyFlow. Compared to consistency-based methods (e.g., rCM [9] for bidirectional video diffusion and Self-Forcing [10] for causal video diffusion), AnyFlow achieves strong performance in the few-step regime and scales effectively with increased sample steps. Compared to the teacher model, AnyFlow preserves test-time scaling ability while significantly improving the full trajectory. Readers can click and play the video clips in this figure using Adobe Acrobat.

Specifically, AnyFlow contains two complementary stages. In the first stage, we develop an improved forward flow map training recipe to convert pretrained video diffusion models into flow map models, providing a strong initialization for any-step generation. However, forward flow map training alone cannot fully remove test-time errors: discretization error remains pronounced under low-NFE sampling, and exposure bias is still severe in causal generation. In the second stage, we introduce on-policy flow map distillation to optimize reverse divergence on model rollouts to mitigate test-time errors. The core design is flow map backward simulation, which replaces expensive full-trajectory simulation with shortcut decomposition, enabling efficient training of intermediate transitions over different time ranges. By combining forward flow map training with reverse-divergence supervision during on-policy distillation, AnyFlow achieves strong few-step quality while continuing to improve with more sampling steps. Moreover, because AnyFlow preserves the fine-grained flow field, the distilled model can be further adapted to downstream datasets through continued fine-tuning.

We validate AnyFlow on both bidirectional and causal video diffusion models, ranging from 1.3B to 14B parameters. Across these settings, AnyFlow matches or surpasses consistency-based counterparts even with few steps and continues to improve as the number of sampling steps increases. In the causal setting, a single AnyFlow model jointly supports text-to-video, image-to-video, and video-to-video generation. For text-to-video, AnyFlow-FAR reaches 84.05 at 4 NFEs and further improves to 84.41 at 32 NFEs on the 14B model, surpassing Krea-Realtime-14B (83.25 at 4 NFEs). For image-to-video, AnyFlow-FAR achieves an 87.87 VBench-I2V score at 4 NFEs, comparable to Wan2.1-I2V-14B using 50×2 NFEs (87.71). In the 14B bidirectional text-to-video setting, AnyFlow reaches 84.04 at 4 NFEs, outperforming rCM-14B (83.73 at 4 NFEs). Our contributions are summarized as

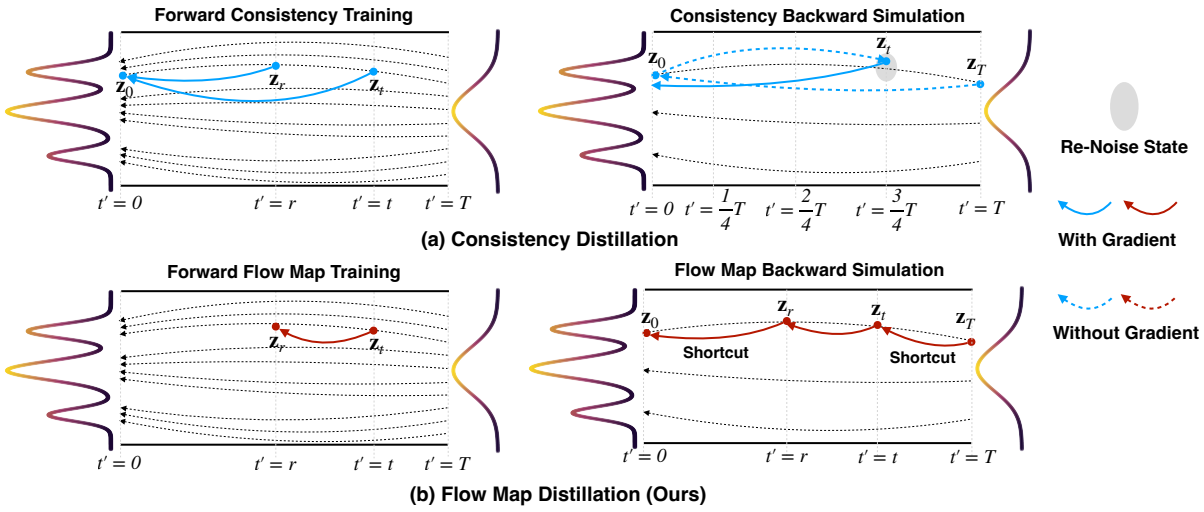


FIGURE 2 Comparison of Distillation Paradigms. (a) Consistency distillation learns a fixed-point mapping from z_t to z_0. Its backward simulation requires re-noising intermediate states, which causes trajectory drift under multi-step sampling. (b) Flow map distillation learns transitions from z_t to z_r. Its backward simulation decomposes Euler sampling trajectories into shortcut segments and enables efficient simulation of transitions across different step sizes.

follows:

- We introduce AnyFlow, the first any-step video diffusion distillation framework built on flow maps, enabling a single model to support arbitrary inference budgets.
- We propose flow map backward simulation for on-policy flow map distillation to mitigate test-time errors (i.e., discretization error and exposure bias) through efficient trajectory decomposition.
- We validate AnyFlow across architectures and model scales, establishing it as a scalable solution for high-quality, any-step video generation.

2 Related Work

2.1 Consistency Models

Consistency Models (CMs) [6] accelerate diffusion sampling by directly learning a mapping from z_t to z_0 along the probability-flow ODE (PF-ODE) defined by a teacher model. For multi-step generation, CMs repeatedly inject noise into intermediate states and perform iterative denoising. Subsequent studies improve CM training stability through annealed time schedules [7, 14] and segmented consistency objectives [15–17]. sCM [8] further simplifies these design choices and provides a strong practical baseline. Building on sCM, rCM [9] introduces score distillation as a regularizer, achieving strong performance in both image and video diffusion distillation. In contrast to prior CM-based approaches, we explore flow map distillation to enable any-step video generation.

2.2 Flow Map Models

Flow Map Models [11,12,18] offer a unified perspective on diffusion modeling and diffusion distillation by learning a generalized transition operator $f_\theta(z_t, t, r)$ between arbitrary points along PF-ODE trajectories. From this perspective, the formulation recovers consistency modeling [6] when $r = 0$ and reduces to standard flow matching [19,20] when $t = r$. However, training flow map models is challenging because it requires accurate transition learning across arbitrary time pairs. MeanFlow [13] is a representative approach that connects instantaneous and average velocities, but it relies on Jacobian-vector products (JVPs), which are difficult to scale under Fully Sharded Data Parallel (FSDP). To overcome this limitation, subsequent studies either approximate JVP terms numerically [21] or derive JVP-free algebraic formulations [22,23]. Building on these advances, AnyFlow leverages flow map distillation for scalable any-step video diffusion, enabling efficient and flexible inference for high-fidelity generation.

2.3 Video Diffusion Distillation

As shown in table 1, recent video diffusion distillation methods typically follow a two-stage pipeline: (1) forward training for initialization and (2) on-policy distillation for refinement. The first stage improves optimization stability, while the second reduces rollout drift during few-step inference. For example, rCM [9] initializes from sCM [8] and then applies consistency backward simulation [24] with distribution matching [25]. In causal video generation, Self-Forcing [10] adopts data-free consistency ODE initialization [26] before performing a similar on-policy refinement stage. Our method follows the same high-level pipeline but replaces consistency modeling with a flow map formulation, enabling stronger any-step performance.

A concurrent work, TMD [27], also studies a flow map formulation for bidirectional video diffusion distillation. The main difference lies in how efficient rollout is achieved. TMD improves rollout efficiency from an architectural perspective by sharing the backbone and introducing an additional flow head. In contrast, our method improves simulation efficiency from the flow-trajectory perspective by decomposing a full trajectory into shortcut transitions based on the composition property of flow maps. As a result, our approach naturally supports arbitrary step budgets and generalizes to both bidirectional and causal architectures.

Causal Video Diffusion. A growing line of work studies causal video diffusion models [28–31]. These models often suffer from exposure bias, which leads to error accumulation during autoregressive generation. Self-Forcing [10] mitigates this issue through on-policy distillation, but its consistency-based formulation is mainly tailored to few-step settings for rollout efficiency. Another line of work [32] focuses more on multi-step models by explicitly modeling test-time errors during pretraining, but it is not directly designed for efficient few-step sampling. By contrast, our method starts from a flow map formulation and uses flow map backward simulation to support both few-step and multi-step causal video generation within a unified framework.

Method	Forward Training	On-Policy Distillation	Causal	Bidirectional	Any-Step
APT1&2 [36,37]	Consistency Training	One-Step Backward Simulation – GAN Loss	✓	✓	✗
rCM [9]	Consistency Training	Consistency Backward Simulation – DMD Loss	✗	✓	✗
Self-Forcing [10]	Consistency ODE Init	Consistency Backward Simulation – DMD Loss	✓	✗	✗
AnyFlow	Flow Map Training	Flow Map Backward Simulation – DMD Loss	✓	✓	✓

TABLE 1 Comparison of Video Diffusion Distillation Methods. Unlike prior consistency-based methods, AnyFlow is built on flow map modeling and supports both causal and bidirectional any-step generation.

2.4 On-Policy Distillation

On-policy distillation mitigates test-time errors by training the student on its own generated trajectories and supervising them with a stronger teacher. This approach has demonstrated benefits in both large language models [33–35] and diffusion models [10, 24, 25].

For diffusion models, on-policy distillation is typically implemented via Distribution Matching Distillation (DMD) [24, 25] or adversarial distillation [36, 37]. Self-Forcing [10] combines self-rollouts with distribution matching against a bidirectional teacher, while APT2 [37] evaluates one-step video rollouts using a discriminator. Our method follows the same on-policy distillation paradigm but introduces flow map backward simulation, specifically designed for the flow map formulation, enabling any-step video generation.

3 Preliminary

3.1 Flow Map Formulation

We first introduce the basic flow map formalism used in AnyFlow. Let z_t denote the latent state at continuous time $t \in [0, 1]$ under the probability-flow ODE (PF-ODE):

$$\frac{dz_t}{dt} = v(z_t, t), \tag{1}$$

where v is the velocity field.

Define the exact flow map $\Phi_{r \leftarrow t}$ of this ODE as the operator that transports states from time t to time r, i.e., $\Phi_{r \leftarrow t}(z_t) = z_r$ for $1 \geq t \geq r \geq 0$. It satisfies two standard properties: identity $\Phi_{t \leftarrow t}(z) = z$ and composition $\Phi_{q \leftarrow r} \circ \Phi_{r \leftarrow t} = \Phi_{q \leftarrow t}$ for $t \geq r \geq q$.

In practice, a neural flow map model learns an approximation

$$f_\theta(z_t, t, r) \approx z_r, \quad 1 \geq t > r \geq 0, \tag{2}$$

with boundary condition $f_\theta(z_t, t, t) = z_t$. Compared with endpoint-only mappings, this parameterization models transitions between arbitrary time pairs, which naturally supports variable step sizes and any-step inference.

MeanFlow Objective. MeanFlow [13] is a representative algorithm for training flow map models. It parameterizes the averaged transport velocity on $[r, t]$ and approximates the

Method	NFEs	Bidirectional Video Diffusion			Causal Video Diffusion		
		Quality	Semantic	Overall	Quality	Semantic	Overall
Forward Training							
Flow Matching Training	4×2	77.82	61.91	74.64	79.05	67.79	76.80
	32×2	85.48	77.24	83.83	85.21	76.65	83.50
Consistency ODE-Init [10, 26]	4	83.01	70.13	80.44	76.99	61.88	73.97
	32	84.78	75.15	82.86	79.73	68.81	77.55
Flow Map Training	4	84.39	71.20	81.75	82.80	71.16	80.48
	32	85.35	75.63	83.40	85.23	74.71	83.13
Forward Training – On-Policy Distillation							
Consistency ODE-Init – Consistency Backward Simulation [10]	4	84.77	75.71	82.96	84.37	74.97	82.49
	32	83.70	64.18	79.80	82.43	68.48	79.64
Flow Map Training – Consistency Backward Simulation	4	85.47	75.88	83.55	84.99	74.97	82.99
	32	85.15	74.19	82.96	85.67	74.78	83.49
Flow Map Training – Flow Map Backward Simulation (Ours)	4	85.24	76.41	83.48	85.60	75.30	83.54
	32	85.70	76.99	83.96	85.92	76.12	83.96

TABLE 2 Quantitative ablation of key designs in AnyFlow. Flow map training provides a stronger initialization for few-step sampling than flow matching training and consistency ODE-Init. However, forward training alone still suffers from test-time errors, which are further mitigated by the on-policy distillation stage. Among the evaluated designs, flow map backward simulation delivers the strongest performance in both few-step and multi-step settings.

transition as $f_\theta(z_t, t, r) = z_t - (t-r)\,u_\theta(z_t, r, t)$. Following this formulation, we optimize u_θ by:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\left\| u_\theta(z_t, r, t) - \mathrm{sg}(u_{\mathrm{tgt}}) \right\|_2^2\right], \tag{3}$$

where $u_{\mathrm{tgt}} = v(z_t, t) - (t-r)\,\frac{du_\theta(z_t,r,t)}{dt}$. Here, $\mathrm{sg}(\cdot)$ denotes stop-gradient.

Differential Derivation Equation. Computing the Jacobian-vector product (JVP) for the derivative term $\frac{du(z_t,r,t)}{dt}$ is expensive and not fully compatible with Fully Sharded Data Parallel (FSDP). Transition Model [21] proposes to approximate this derivative using the finite difference method:

$$\frac{d}{dt}u(z_t, r, t) \approx \frac{u(z_{t+\Delta t}, r, t+\Delta t) - u(z_{t-\Delta t}, r, t-\Delta t)}{2\Delta t}, \tag{4}$$

This approximation requires only two forward passes per training step and is compatible with FSDP.

3.2 On-Policy Distillation

On-policy distillation is designed to reduce train–test mismatch by training the student on its own rollout states while being guided by a strong teacher. In diffusion models, this

FIGURE 3 Qualitative Ablation of On-Policy Distillation. (a) Forward flow map training alone still suffers from test-time errors, including discretization error and exposure bias. (b) On-policy flow map distillation optimizes reverse divergence on model self-rollouts to mitigate these test-time errors. Readers can click and play the video clips in this figure using Adobe Acrobat.

paradigm is mainly instantiated as Distribution Matching Distillation (DMD). It typically consists of two key components: backward simulation and a distribution-matching objective.

Backward Simulation. Given noise z, the student first produces a sample through its inference trajectory, $\hat{z}_0 = f_\theta(z)$. This self-generated trajectory is then used to compute the distillation loss, which helps mitigate exposure bias and discretization error at test time.

Distribution Matching Objective. We follow DMD-style reverse-KL training [24, 25]. First, DMD re-noises the self-rollout sample $\hat{z}_0$ at $t \in [0, T]$ as $z_t = (1-t)\hat{z}_0 + t\epsilon 0$, where $\epsilon 0 \sim \mathcal{N}(0, I)$. It then estimates the DMD gradient as follows:

$$\nabla_\theta \mathcal{L}_{\text{DMD}} = -\mathbb{E}_{t,z}\left[\left(s_{\text{real}}(z_t, t) - s_{\text{fake}}(z_t, t)\right)\frac{\partial f_\theta(z)}{\partial \theta}\right], \tag{5}$$

where s_{real} and s_{fake} are the real and fake score functions, respectively.

4 Method

4.1 AnyFlow Motivation

As summarized in table 1, most recent few-step video distillation methods are built on consistency models. Although effective under very small sampling budgets, their performance often degrades as the number of sampling steps increases (fig. 1). The underlying reason is that consistency-style sampling repeatedly performs endpoint projection and re-noising, which gradually drives the trajectory away from the target PF-ODE path during multi-step inference.

Motivation. Our goal is to preserve the strong few-step efficiency of distilled video diffusion models while enabling quality to improve, rather than deteriorate, as more sampling steps are used. To this end, AnyFlow shifts the distillation target from fixed-point endpoint mapping to flow map transition learning, i.e., $f_\theta : (z_t, t, r) \mapsto z_r$. Instead of only predicting z_0, the model learns transitions between arbitrary time pairs. This formulation naturally supports both small time gaps for stable local refinement and large time gaps for efficient long-range jumps, making it well suited for any-step generation.

Challenge. Forward flow map training alone (e.g., MeanFlow [13]) is insufficient for strong test-time performance in video generation. As shown in section 3.2 and table 2, it still suffers from test-time errors, especially discretization error in few-step sampling and exposure bias in causal generation. This motivates an additional on-policy distillation stage to correct rollout mismatch. However, designing such a stage for flow map models is non-trivial. Unlike consistency models, which can naturally reach z_0 at intermediate

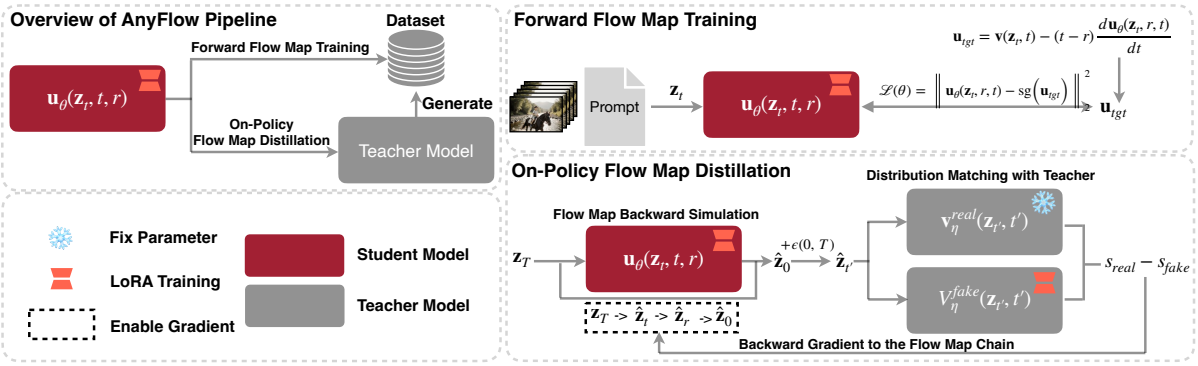


FIGURE 4 Overview of the AnyFlow Pipeline. AnyFlow enables any-step video generation by jointly learning forward flow map training from synthetic data and on-policy distillation with flow map backward simulation under teacher guidance.

steps and therefore allow KL gradients to be computed directly at endpoint states, flow map models require a new simulation strategy that can accommodate transitions between arbitrary time pairs while remaining efficient for training.

4.2 AnyFlow Pipeline

Overview. As illustrated in fig. 4, AnyFlow is trained in two complementary stages: forward flow map training (section 4.2.1) and on-policy distillation (section 4.2.2). In Stage 1, we fine-tune a pretrained video diffusion model on teacher-synthesized data to learn a stable transition operator $f_\theta : (z_t, t, r) \mapsto z_r$, which provides a strong initialization for any-step generation. In Stage 2, starting from the converged Stage-1 checkpoint, we jointly optimize forward flow map training and DMD-based on-policy distillation. This stage uses flow map backward simulation to generate student rollouts and apply reverse-divergence supervision from a strong teacher, thereby reducing discretization error and exposure bias while preserving any-step capability.

4.2.1 Flow Map Training

We employ the MeanFlow objective (eq. (3)) alongside the differential derivation equation (eq. (4)) as our primary training framework. Because the original MeanFlow architecture is optimized for pretraining, directly adapting it to post-training for video diffusion models requires several additional design choices for strong performance. To this end, we introduce several modifications: interpolated timestep conditioning, time sampling, guidance-fused training, and adaptive loss reweighting.

Interpolated Timestep Conditioning. Converting a pretrained diffusion model into a flow map model requires introducing an additional timestep, r. A natural choice is to follow MeanFlow [13] and use $\mathrm{emb}(t) + \mathrm{emb}'(t - r)$. Prior work such as TMD [27] further applies a zero-initialized output projection to emb' for a smooth start in post-training. However, we find this design unstable in our setting: it tends to produce timestep embeddings with much larger norms than the pretrained ones, leading to over-saturated generation results,

Algorithm 1: AnyFlow: Forward Flow Map Training

```
# fn(z, c, t, r): model to predict u
# x: training batch
# c: text embedding
# g: CFG scale

t, r = sample_t_r()   # t > r
e = randn_like(x)

z = (1 − t) ∗ x + t ∗ e
v = e − x

# Guidance-Fused Training
u_c = fn(z, c, t, r)
u_∅ = fn(z, ∅, t, r)
u = 1/g (u_c − (1 − g) ∗ stopgrad(u_∅))

# Differential Derivation Equation
z⁺ = z + εv
z⁻ = z − εv
du/dt = (fn(z⁺, c, t+ε, r) − fn(z⁻, c, t−ε, r)) / 2εg
u_tgt = v − (t − r) du/dt

loss = metric(u − stopgrad(u_tgt))
```

Algorithm 2: AnyFlow: On-Policy Flow Map Distillation

```
# fn(z, c, t, r): model to predict u
# x: training batch
# c: text embedding

while true do
    # Sample Inference Step
    s ∼ Uniform(1, 2, ..., T)

    # Sample Gradient Step
    t ∼ Uniform(1, 2, ..., s)
    r = t − T/s

    # Sample Initial Noise
    z_T = randn_like(x)

    # Flow Map Backward Simulation
    z_t = fn(z_T, c, T, t) # With Grad
    z_r = fn(z_t, c, t, r) # With Grad
    z_0 = fn(z_r, c, r, 0) # With Grad

    Update θ via distribution matching
      loss using z_0 and renoise at [0, T]
end
```

as shown in fig. 11.

To address this issue, we instead use an interpolated timestep conditioning scheme:

$$g \cdot \mathrm{emb}(t) + (1 - g) \cdot \mathrm{emb}'(r),$$

where emb' is initialized from the pretrained emb and g is fixed to 0.25 in all experiments. At the beginning of training, this formulation reduces to the pretrained timestep embedding in the boundary case $t = r$. As verified in fig. 11, interpolated timestep conditioning keeps the embedding norm well aligned with the pretrained model and suppresses over-saturated results observed in zero-init timestep conditioning.

Time Sampler. Following MeanFlow [13], we first sample t and r from a uniform distribution and then reorder them as $t = \max(t, r)$ and $r = \min(t, r)$. We further introduce a reweighting function $w(t)$ based on the sampled timestep t to balance the training objective across different noise levels. Different choices of $w(t)$ are discussed in the fig. 10.

Guidance-Fused Training. We also incorporate classifier-free guidance (CFG) into flow map training following MeanFlow [13], which allows CFG to be omitted at test time for

faster inference. Unlike MeanFlow, which fuses the CFG objective into the target velocity field u_{tgt}, we fuse it into the prediction to better align with the guidance scale defined by the pretrained diffusion model:

$$u = \frac{1}{g}\left(u_c - (1-g)\,\mathrm{sg}(u_\varnothing)\right), \tag{6}$$

where $\mathrm{sg}(\cdot)$ denotes the stop-gradient operation and $u_\varnothing$ is the model prediction with null conditioning.

Adaptive Loss Reweighting. To stabilize flow map training, we introduce an adaptive loss reweighting scheme. Unlike MeanFlow [13], which uses a weighting function $w = 1/(\|\Delta\|_2^2 + c)$ for pre-training, where Δ denotes the regression loss and c is a small constant, our approach is designed for post-training, where a reliable instantaneous velocity field has already been established at $t = r$. We use the loss at $t = r$ as a baseline to dynamically scale the loss for other time steps $t \neq r$. In each iteration, we sample 50% of the batch from the boundary cases ($t = r$) and define the adaptive weight $w_{t,r}$ as:

$$w_{t,r} = \begin{cases} 1, & t = r \\ \dfrac{\mu_{t=r}}{\|\Delta\|_2^2 + c}, & t \neq r \end{cases} \tag{7}$$

where $\mu_{t=r} = \mathbb{E}_{t=r}[\|\Delta\|_2^2]$ is the average regression loss over the boundary samples. This formulation aligns the loss magnitude at $t \neq r$ with the well-optimized field at $t = r$, thereby preserving the learned instantaneous velocity and preventing gradient instability.

Discussion. The full algorithm of our improved forward flow map training is summarized in algorithm 1. As shown in table 2, compared with standard flow matching, flow map training achieves comparable multi-step performance while delivering significantly better few-step performance. Compared with Consistency ODE-Init, it achieves stronger multi-step performance while maintaining comparable few-step quality. Overall, flow map training provides a strong initialization for the subsequent on-policy distillation stage.

4.2.2 On-Policy Flow Map Distillation

Although forward flow map training provides a strong initialization for any-step generation, it still suffers from test-time errors, especially discretization error at low NFEs and exposure bias in causal generation, as shown in section 3.2. To correct rollout drift, we further introduce on-policy distillation with teacher guidance. Following prior work [9, 10], we instantiate on-policy diffusion distillation with distribution matching distillation (DMD), which requires the student to perform a self-rollout (i.e., backward simulation) to z_0 before re-noising in order to compute the Kullback–Leibler (KL) gradient. Below, we describe our design of backward simulation for flow map models.

Limitations of Consistency Backward Simulation. Existing backward simulation methods for DMD [9, 10] largely follow the logic of consistency models, as illustrated in fig. 5(a). This design is convenient for on-policy distillation because the consistency scheduler can reach the z_0 state at intermediate steps, making endpoint-based supervision straightforward. In practice, gradients are often truncated before the target gradient step to avoid the memory

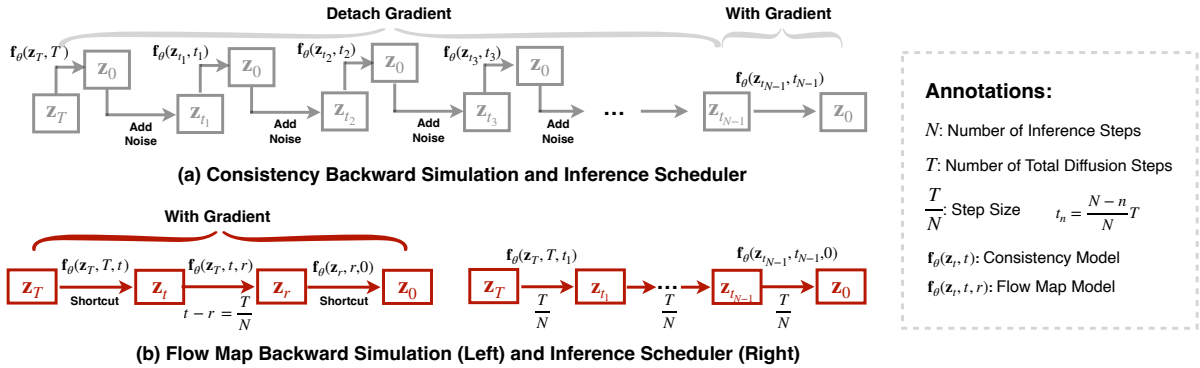


FIGURE 5 Comparison of Backward Simulation Paradigms. (a) Consistency backward simulation and sampling both follow consistency trajectories, where truncated gradients are used to reduce training cost of on-policy distillation. (b) Flow map backward simulation enables efficient simulation of Euler sampling trajectories. It decomposes the full rollout trajectory into shortcut segments to reduce the training cost of on-policy distillation.

[width=]8figures/videosrc/ablation_anyflow_simulation/0000000020

FIGURE 6 Qualitative Ablation of Backward Simulation. (a) Consistency backward simulation exhibits over-smoothed textures and degraded motion as more sampling steps are allocated. (b) Flow map backward simulation produces a straighter trajectory, enabling test-time scaling while maintaining strong few-step quality. Readers can click and play the video clips in this figure using Adobe Acrobat.

cost and instability of backpropagating through a long rollout chain. In inference time, the same consistency sampler is then reused.

However, unlike ODE trajectories learned through flow-matching pretraining, consistency samplers require additional re-noising to obtain intermediate states during multi-step sampling. These re-noised states deviate from the base PF-ODE trajectory, which limits generalization beyond the simulation step used during simulation. Moreover, simulating arbitrary step counts with consistency backward simulation is computationally expensive because it requires rolling out the full trajectory.

Flow Map Backward Simulation. Unlike consistency backward simulation, our design directly exploits the learned transition capability of flow map models. Since a flow map model is able to predict transitions between arbitrary time pairs, it can naturally shortcut long rollout trajectories instead of explicitly simulating every intermediate step, based on the composition property of flow maps:

$$\mathrm{f}_\theta(\mathrm{z}_t, t, q) \approx \mathrm{f}_\theta\big(\mathrm{f}_\theta(\mathrm{z}_t, t, r), r, q\big), \quad t > r > q. \tag{8}$$

Concretely, for a target sampling budget of N steps, we first choose an intermediate timestep t along the sampled trajectory and then set the next timestep r such that $t - r = \frac{T}{N}$. As illustrated in fig. 5, this decomposes a rollout trajectory from T to 0 into three segments: $T \to t, t \to r$, and $r \to 0$. The first and last segments, $T \to t$ and $r \to 0$, are handled by shortcut transitions of the learned flow map, while $t \to r$ is the target transition along the sampled

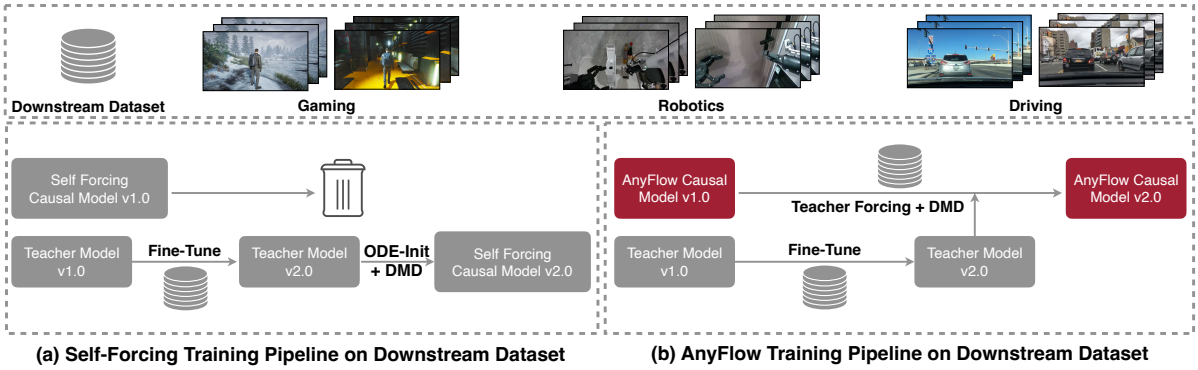


FIGURE 7 Illustration of the AnyFlow Fine-Tuning Pipeline for Downstream Applications. Unlike self-forcing pretrained causal models that are difficult to adapt to new downstream datasets, AnyFlow supports continued training. This capability bypasses the complexities of retraining a causal generator.

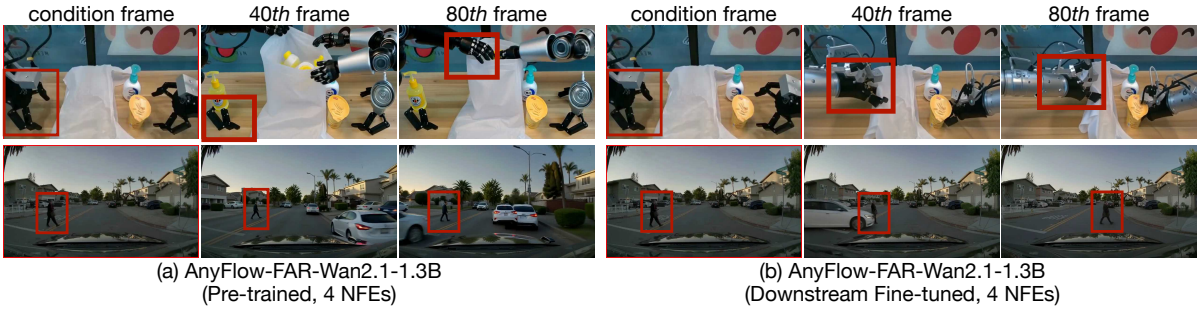


FIGURE 8 Visualization of the AnyFlow Fine-Tuning for Downstream Applications. While the pre-trained AnyFlow-FAR-Wan2.1-1.3B struggles with identity preservation (e.g., robot arm type) and trajectory accuracy (e.g., moving pedestrians) in specialized domains, AnyFlow supports continued training on downstream datasets, allowing for efficient application-specific tuning without full retraining.

trajectory. Once the KL gradient is computed at z_0, it is backpropagated through the whole chain. By varying N during training, we can efficiently simulate different inference step budgets with the same computation cost.

After on-policy distillation, we can simply use a standard Euler scheduler for sampling. Because flow map backward simulation trains transitions over different time intervals, the same model naturally supports different inference budgets at test time.

Discussion. The full algorithm of our on-policy distillation stage is shown in algorithm 2. As shown in table 2, Consistency ODE-Init combined with consistency backward simulation exhibits the weakest test-time scaling, especially at 32 NFEs. Replacing the forward training stage with flow map training narrows this gap, but noticeable degradation still remains under multi-step sampling. By further adopting our flow map backward simulation, both few-step and multi-step performance are improved through on-policy distillation. As shown in section 4.2.1, flow map backward simulation produces a straighter trajectory, which leads to more consistent improvement as the sampling budget increases, whereas

[width=]8figures/videosrc/t2v14b_causal/0000000020

(a) Text-to-video comparison of causal video diffusion model (14B).

[width=]8figures/videosrc/t2v14b_bidirectional/0000000020

(b) Text-to-video comparison of bidirectional video diffusion model (14B).

[width=]8figures/videosrc/i2v14b/0000000020

(c) Image-to-video comparison of video diffusion model (14B).

FIGURE 9 Qualitative Comparison of AnyFlow and Baselines at the 14B Model Scale. Readers can click and play the video clips in this figure using Adobe Acrobat.

consistency-based simulation degrades at 32 NFEs.

4.3 AnyFlow Application

AnyFlow for Bidirectional Video Diffusion. For bidirectional video diffusion, we simply follow the established AnyFlow pipeline, as detailed in algorithm 1 and algorithm 2.
AnyFlow for Causal Video Diffusion. For causal video diffusion, we adopt the FAR [38] training pipeline, which uses context compression with asymmetric patchify kernels to encode videos efficiently. Specifically, we keep three full-token chunks with the standard patchify kernel size of 2, while compressing the remaining chunks with a larger kernel size of 4. This design substantially reduces teacher-forcing training cost and KV-cache size during sampling. To jointly support Image-to-Video (I2V) and Text-to-Video (T2V), we use a non-uniform chunk partition: the first chunk has size 1 for precise first-frame conditioning, and subsequent chunks use size 3 to balance motion modeling and throughput. The training objective remains the same as in the bidirectional setting, and we additionally cache and reuse KV states during rollouts to improve simulation efficiency for causal video diffusion models.
Continued Training on Downstream Datasets. A practical advantage of AnyFlow is that it preserves the pretrained model's instantaneous flow field, making the distilled model compatible with continued training on downstream datasets. As illustrated in fig. 7, AnyFlow enables continued training from a pretrained checkpoint, unlike Self-Forcing, where further training of the distilled model is difficult.

This property is especially useful in specialized domains whose motion patterns and visual statistics differ from those of the original training corpus. As shown in fig. 8, the base distilled model, AnyFlow-FAR-Wan2.1-1.3B, still struggles with identity preservation in robotics videos and trajectory accuracy in driving scenes. After continued training on downstream data, these errors are substantially reduced.

5 Experiments

Model	#Params	Resolution	NFEs	Evaluation Scores ↑		
				Quality	Semantic	Total
Bidirectional Video Diffusion Models						
LTX-Video [5]	1.9B	512 × 768	40×2	82.30	70.79	80.00
CogVideoX-2B [4]	2B	480 × 720	50×2	82.48	77.81	81.55
HunyuanVideo [3]	13B	720 × 1280	50×2	85.09	76.88	83.24
†Wan2.1-T2V-1.3B [1]	1.3B	480 × 832	50×2	84.99	76.23	83.24
†rCM-Wan2.1-T2V-1.3B [9]	1.3B	480 × 832	4	84.71	73.74	82.51
†AnyFlow-Wan2.1-T2V-1.3B	1.3B	480 × 832	4	85.24	76.41	83.48
†AnyFlow-Wan2.1-T2V-1.3B	1.3B	480 × 832	32	85.70	76.99	83.96
†Wan2.1-T2V-14B [1]	14B	480 × 832	50×2	85.77	75.58	83.74
†rCM-Wan2.1-T2V-14B [9]	14B	480 × 832	4	85.47	76.72	83.73
†AnyFlow-Wan2.1-T2V-14B	14B	480 × 832	4	85.70	77.38	84.04
†AnyFlow-Wan2.1-T2V-14B	14B	480 × 832	32	85.76	77.44	84.10
Causal Video Diffusion Models						
MAGI-1 [39]	4.5B	720 × 720	8	80.98	65.68	77.92
SkyReels-V2 [40]	1.3B	540 × 960	50×2	84.70	74.53	82.67
†Self-Forcing [10]	1.3B	480 × 832	4	85.23	76.01	83.39
†AnyFlow-FAR-Wan2.1-1.3B	1.3B	480 × 832	4	85.60	75.30	83.54
†AnyFlow-FAR-Wan2.1-1.3B	1.3B	480 × 832	32	85.92	76.12	83.96
†LightX2V-Wan2.1-14B-CausVid [41]	14B	480 × 832	9	85.29	75.96	83.42
†FastVideo-CausalWan2.2-A14B-Preview [42]	14B	480 × 832	8	84.28	78.49	83.12
†Krea-Realtime-Wan2.1-14B [43]	14B	480 × 832	4	84.80	77.07	83.25
†AnyFlow-FAR-Wan2.1-14B	14B	480 × 832	4	85.82	76.97	84.05
†AnyFlow-FAR-Wan2.1-14B	14B	480 × 832	32	86.12	77.55	84.41

TABLE 3 Text-to-Video Evaluation on VBench. † indicates that we re-evaluate model performance using the official VBench augmented prompts under the same setting. Results for all other models are taken directly from their respective papers.

5.1 Implementation Details

We implement AnyFlow on top of Wan2.1 [1] in the Diffusers framework [45]. We train the model on a synthetic dataset of 256K prompt–video pairs generated from Wan2.1-T2V-14B, where each sample contains up to 81 frames at a resolution of 480 × 832. Training is performed in two stages. In Stage 1, we optimize the forward flow map objective using AdamW [46] with a learning rate of 5×10^{-5} and a global batch size of 32 for the 1.3B model and 16 for the 14B model, for 6,000 and 4,000 iterations, respectively. In Stage 2, we perform on-policy distillation by jointly optimizing the forward flow map and on-policy objectives with AdamW at a learning rate of 2×10^{-6} for 800 iterations. In both stages, we use parameter-efficient fine-tuning with LoRA [47] of rank 256.

Model	#Params	Resolution	NFEs	Evaluation Scores ↑		
				Quality	I2V	Total
CogVideoX-5B-I2V [4]	5B	480 × 720	50×2	78.61	94.79	86.70
HunyuanVideo-I2V [3]	13B	720 × 1280	50×2	78.54	95.10	86.82
Step-Video-TI2V [44]	30B	540 × 960	50×2	81.22	95.50	88.36
MAGI-1 [39]	24B	720 × 1280	32×2	82.44	96.12	89.28
†Wan2.1-I2V-14B [1]	14B	480 × 832	50×2	80.30	95.12	87.71
†FastVideo-CausalWan2.2-A14B-Preview [42]	14B	480 × 832	8	78.82	94.81	86.82
†AnyFlow-FAR-Wan2.1-14B	14B	480 × 832	4	80.39	95.35	87.87

TABLE 4 Image-to-Video Evaluation on VBench-I2V. † indicates that we re-evaluate model performance under the same setting.

5.2 Evaluation Setting

For text-to-video (T2V) evaluation, we adopt VBench [48], which evaluates generation quality across 16 fine-grained dimensions grouped into two high-level categories: Quality score and Semantic score. For image-to-video (I2V) evaluation, we use VBench-I2V [49] and report both the Quality score and the I2V score.

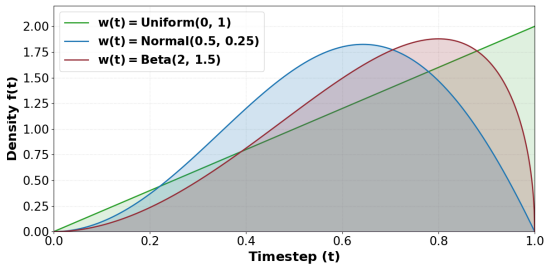
To ensure a fair comparison, we re-evaluate key counterparts under a unified protocol using the official VBench augmented prompts. These counterparts include the Wan2.1 base model [1], Self-Forcing [10], and rCM [9], as well as community-trained 14B causal models including FastVideo [42], Krea-Realtime-14B [43], and LightX2V [41]. Results for all other methods are taken directly from their original papers.

5.3 Main Results

Quantitative Comparison. As shown in table 3, AnyFlow achieves strong few-step performance and continues to improve as the sampling budget increases. On the 14B bidirectional backbone, AnyFlow-Wan2.1-T2V-14B obtains 84.04 at 4 NFEs, outperforming rCM-Wan2.1-T2V-14B (83.73 at 4 NFEs). On the 14B causal backbone, AnyFlow-FAR-Wan2.1-14B reaches 84.05 at 4 NFEs and further improves to 84.41 at 32 NFEs, outperforming strong community counterparts.

For image-to-video results in table 4, AnyFlow-FAR-Wan2.1-14B achieves a VBench-I2V score of 87.87 using only 4 NFEs. This result is comparable to Wan2.1-I2V-14B with 50×2 NFEs (87.71) and surpasses FastVideo-CausalWan2.2-A14B-Preview (86.82), showing that AnyFlow improves both visual quality and I2V faithfulness while retaining strong sampling efficiency.

Qualitative Comparison. As shown in fig. 9, we provide qualitative comparisons at the 14B scale for both T2V and I2V generation. In causal T2V examples, FastVideo-Wan2.2-A14B-Preview exhibits blurry details, LightX2V-Wan2.1-14B-CausVid shows flickering artifacts, and Krea-Realtime-14B produces unrealistic motion in some clips. By comparison,



(a) Timestep density $f(t) = p(t) \cdot w(t)$, where $p(t) = 2t$.

w(t)	Bidirectional		Causal	
	4 NFEs	32 NFEs	4 NFEs	32 NFEs
Uniform$(0, 1)$	83.46	83.75	83.58	83.91
Normal$(0.5, 0.25)$	83.40	83.79	83.54	83.93
Beta$(2, 1.5)$	83.48	83.96	83.54	83.96

(b) Quantitative ablation (Vbench overall score) of the loss weight function $w(t)$.

FIGURE 10 Ablation Study of Loss Weight Function $w(t)$. Among the different weighting strategies, $w(t) = \text{Beta}(2, 1.5)$ demonstrates the best performance across both architectures.

AnyFlow-FAR-Wan2.1-14B delivers the best visual quality and motion with 4 NFEs. In bidirectional T2V examples, AnyFlow-Wan2.1-T2V-14B shows slightly better motion than rCM-Wan2.1-T2V-14B.

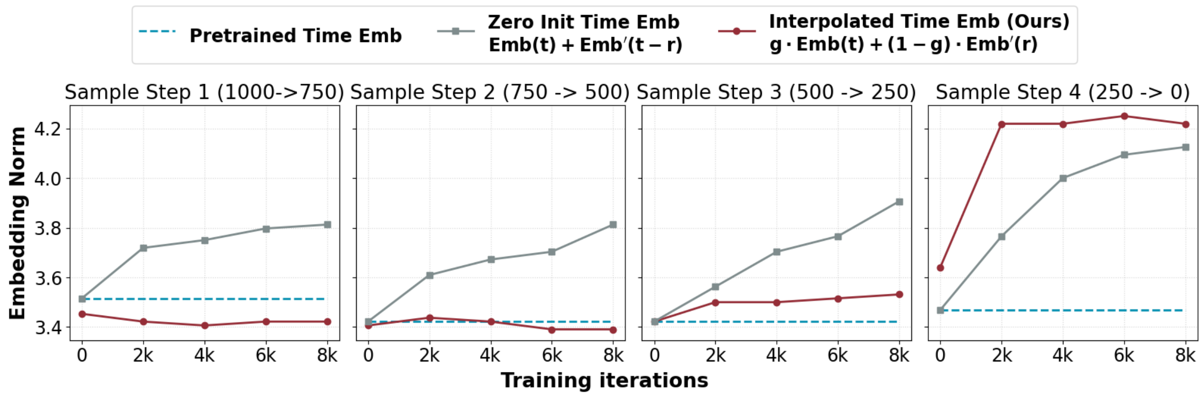
For I2V generation, AnyFlow reuses the causal generator through our non-uniform chunk partition and maintains good first-frame faithfulness with smooth motion transitions. Compared with Wan2.1-I2V-14B, AnyFlow-FAR-Wan2.1-14B shows comparable temporal stability and motion quality, while FastVideo-Wan2.2-A14B-Preview exhibits visible flickering and inconsistencies with the input image. Overall, these qualitative results are consistent with the quantitative trends and further support the effectiveness of AnyFlow in few-step sampling.

5.4 Ablation Study

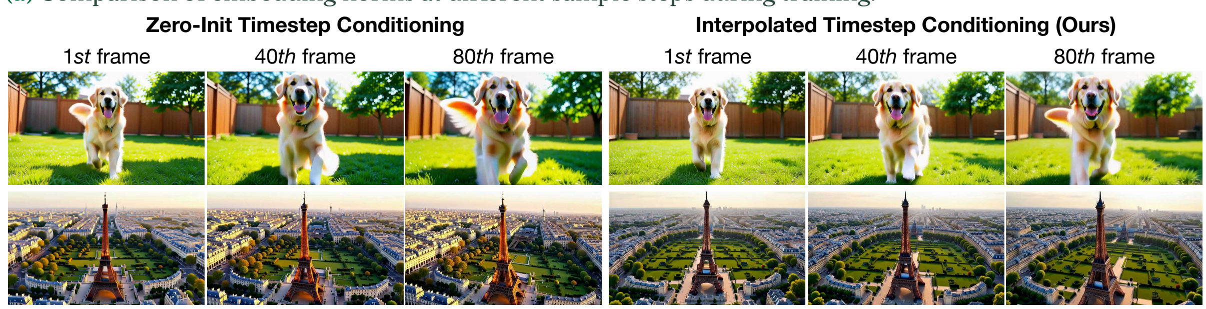
Time Sampler. We investigate different choices of the time sampler, with their induced densities over t visualized in fig. 10a. The uniform weighting yields the worst test-time scalability, as reflected by its 32-step performance in fig. 10b, likely because it mismatches the logit-normal time weighting used in pretraining. Among the other variants, assigning more probability mass to high-noise regions leads to better overall performance. Therefore, we fix $w(t) = \text{Beta}(2, 1.5)$ in all experiments.

Interpolated Timestep Conditioning. Incorporating an additional timestep embedding during post-training is non-trivial. A baseline design is to add a new embedding projection whose final layer is zero-initialized, as in TMD [27]. However, under zero initialization, the model needs to amplify the influence of the r embedding, which causes its norm to keep increasing during training and eventually leads to over-saturated results, as shown in fig. 11. In contrast, our interpolated timestep conditioning preserves the boundary case $t = r$ for stability while avoiding an overly cold start when $t \neq r$. As a result, the embedding norm remains stable after roughly 2K training steps and stays much closer to that of the pretrained model, effectively suppressing the over-saturation effect.

Comparison to ODE-Init. We compare AnyFlow with Consistency ODE-Init for causal video generation in section 5.4. For a fair comparison, we fine-tune the teacher on the same



(a) Comparison of embedding norms at different sample steps during training.



(b) Visual comparison of zero-init timestep conditioning and interpolated timestep conditioning.

FIGURE 11 Ablation Study of Timestep Conditioning. Compared with zero-init timestep conditioning, the proposed interpolated method achieves embedding norms more consistent with pretrained embeddings, preventing over-saturated visual results.

[width=]8figures/videos/ablation_odeinit/0000000020

FIGURE 12 Qualitative Ablation Study of ODE-Init and AnyFlow on Causal Video Generation. Readers can click and play the video clips in this figure using Adobe Acrobat.

synthetic dataset, since we do not have access to the original training data of the Wan base model. After flow map training, AnyFlow produces clearer results than Consistency ODE-Init at both 4 and 32 NFEs, indicating that it better preserves the knowledge inherited from the flow-matching-pretrained base model. After on-policy distillation, AnyFlow further reduces test-time errors in few-step sampling and improves the overall sampling trajectory. We also observe that our method preserves the style and content of the pretrained model more faithfully.

Training Cost Breakdown. table 5 reports the training cost of each component in AnyFlow, measured in seconds per iteration on 8×H100 GPUs. In the forward training stage, guidance-fused training and the differential derivation equation introduce additional overhead compared with standard flow matching, but the overall cost remains practical for large-scale training. In the on-policy distillation stage, our flow map backward simulation incurs a slightly higher cost than consistency backward simulation at 4 steps, specifically 15.7% more for the causal model and 22.5% more for the bidirectional model, because we back-propagate through the full rollout chain. However, our method becomes significantly

Method	Training Cost (s/iter on H100)	
	Causal	Bidirectional
Forward Training		
Standard Flow-Matching	5.8	9.7
– Guidance-Fused Training	7.8	12.3
– Guidance-Fused Training – Differential Derivation Equation	10.4	16.8
On-Policy Distillation		
Consistency Backward Simulation (4 Steps)	45.9	41.8
Flow Map Backward Simulation (4 Steps)	53.1 (–15.7%)	51.2 (–22.5%)
Consistency Backward Simulation (16 Steps)	93.8	96.6
Flow Map Backward Simulation (16 Steps)	53.1 (-43.4%)	51.2 (-47.0%)

TABLE 5 Training Cost Breakdown of AnyFlow. Costs are measured on Wan2.1-1.3B-T2V with batch size 16 on a node with 8×NVIDIA H100 GPUs. During on-policy distillation, we evaluate the upper-bound cost (comprising one generator and one discriminator update) for the simulation trajectories.

more efficient when simulating larger step counts. At 16 steps, it reduces the training cost by 43.4% for the causal model and 47.0% for the bidirectional model compared with consistency backward simulation, thanks to the shortcut transitions learned by the flow map.

6 Conclusion

We present AnyFlow, the first any-step video diffusion distillation framework based on a two-time flow map formulation. By learning transitions between arbitrary time pairs, AnyFlow supports a wide range of sampling budgets within a single model. Built on large pretrained video diffusion backbones, it combines an improved forward flow map training recipe with flow map backward simulation for on-policy distillation, thereby reducing discretization error and exposure bias during sampling. Across both bidirectional and causal architectures, and across model scales from 1.3B to 14B, AnyFlow consistently matches or outperforms strong consistency-based counterparts in the few-step regime while continuing to improve as the sampling budget increases. These results highlight a practical and scalable path toward high-quality any-step video generation.

Limitations. The primary limitation of our method is its reliance on external datasets for flow map training. Even when synthetic data are used, the training distribution may still differ from that of the base model, which can introduce mild distribution shift, such as smoother textures. This issue could be mitigated by applying AnyFlow with the same data used to pretrain the base model.

Future Work. Although AnyFlow establishes the first any-step, flow-map-based video diffusion distillation framework, there is still substantial room for improving the training

methodology. One important direction is to develop more effective and stable forward flow map training strategies to improve robustness across different NFE regimes. In addition, because AnyFlow provides a scalable recipe for learning any-step causal video diffusion from both data and a teacher model, a natural next step is to extend the framework to autoregressive long-video generation with dedicated long-video training.

References

[1] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.

[2] Niket Agarwal, Arslan Ali, Maciej Bala, Yogesh Balaji, Erik Barker, Tiffany Cai, Prithvijit Chattopadhyay, Yongxin Chen, Yin Cui, Yifan Ding, et al. Cosmos world foundation model platform for physical ai. *arXiv preprint arXiv:2501.03575*, 2025.

[3] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, et al. Hunyuanvideo: A systematic framework for large video generative models. *arXiv preprint arXiv:2412.03603*, 2024.

[4] Zhuoyi Yang, Jiayan Teng, Wendi Zheng, Ming Ding, Shiyu Huang, Jiazheng Xu, Yuanming Yang, Wenyi Hong, Xiaohan Zhang, Guanyu Feng, et al. Cogvideox: Text-to-video diffusion models with an expert transformer. *arXiv preprint arXiv:2408.06072*, 2024.

[5] Yoav HaCohen, Nisan Chiprut, Benny Brazowski, Daniel Shalem, Dudu Moshe, Eitan Richardson, Eran Levin, Guy Shiran, Nir Zabari, Ori Gordon, Poriya Panet, Sapir Weissbuch, Victor Kulikov, Yaki Bitterman, Zeev Melumian, and Ofir Bibi. LTX-Video: Realtime video latent diffusion. *arXiv preprint arXiv:2501.00103*, 2024.

[6] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. 2023.

[7] Yang Song and Prafulla Dhariwal. Improved techniques for training consistency models. In *ICLR*, 2024.

[8] Cheng Lu and Yang Song. Simplifying, stabilizing and scaling continuous-time consistency models. *arXiv preprint arXiv:2410.11081*, 2024.

[9] Kaiwen Zheng, Yuji Wang, Qianli Ma, Huayu Chen, Jintao Zhang, Yogesh Balaji, Jianfei Chen, Ming-Yu Liu, Jun Zhu, and Qinsheng Zhang. Large scale diffusion distillation via score-regularized continuous-time consistency. *arXiv preprint arXiv:2510.08431*, 2025.

[10] Xun Huang, Zhengqi Li, Guande He, Mingyuan Zhou, and Eli Shechtman. Self forcing: Bridging the train-test gap in autoregressive video diffusion. In *NeurIPS*, 2025.

[11] Amirmojtaba Sabour, Sanja Fidler, and Karsten Kreis. Align your flow: Scaling continuous-time flow map distillation. *arXiv preprint arXiv:2506.14603*, 2025.

[12] Nicholas M Boffi, Michael S Albergo, and Eric Vanden-Eijnden. Flow map matching with stochastic interpolants: A mathematical framework for consistency models. *arXiv preprint arXiv:2406.07507*, 2024.

[13] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling. *arXiv preprint arXiv:2505.13447*, 2025.

[14] Zhengyang Geng, Ashwini Pokle, William Luo, Justin Lin, and J Zico Kolter. Consistency

models made easy. *arXiv preprint arXiv:2406.14548*, 2024.

[15] Fu-Yun Wang, Zhaoyang Huang, Alexander Bergman, Dazhong Shen, Peng Gao, Michael Lingelbach, Keqiang Sun, Weikang Bian, Guanglu Song, Yu Liu, et al. Phased consistency models. *Advances in neural information processing systems*, 37:83951–84009, 2024.

[16] Yuxi Ren, Xin Xia, Yanzuo Lu, Jiacheng Zhang, Jie Wu, Pan Xie, Xing Wang, and Xuefeng Xiao. Hyper-SD: Trajectory segmented consistency model for efficient image synthesis. *NeurIPS*, 2024.

[17] Sangyun Lee, Yilun Xu, Tomas Geffner, Giulia Fanti, Karsten Kreis, Arash Vahdat, and Weili Nie. Truncated consistency models. *arXiv preprint arXiv:2410.14895*, 2024.

[18] Dongjun Kim, Chieh-Hsin Lai, Wei-Hsiang Liao, Naoki Murata, Yuhta Takida, Toshimitsu Uesaka, Yutong He, Yuki Mitsufuji, and Stefano Ermon. Consistency trajectory models: Learning probability flow ODE trajectory of diffusion. In *ICLR*, 2024.

[19] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*, 2022.

[20] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[21] Zidong Wang, Yiyuan Zhang, Xiaoyu Yue, Xiangyu Yue, Yangguang Li, Wanli Ouyang, and Lei Bai. Transition models: Rethinking the generative learning objective. *arXiv preprint arXiv:2509.04394*, 2025.

[22] Tianze Luo, Haotian Yuan, and Zhuang Liu. Soflow: Solution flow models for one-step generative modeling. *arXiv preprint arXiv:2512.15657*, 2025.

[23] Yi Guo, Wei Wang, Zhihang Yuan, Rong Cao, Kuan Chen, Zhengyang Chen, Yuanyuan Huo, Yang Zhang, Yuping Wang, Shouda Liu, et al. Splitmeanflow: Interval splitting consistency in few-step generative modeling. *arXiv preprint arXiv:2507.16884*, 2025.

[24] Tianwei Yin, Michaël Gharbi, Taesung Park, Richard Zhang, Eli Shechtman, Fredo Durand, and Bill Freeman. Improved distribution matching distillation for fast image synthesis. *NeurIPS*, 2024.

[25] Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Fredo Durand, William T Freeman, and Taesung Park. One-step diffusion with distribution matching distillation. In *CVPR*, 2024.

[26] Tianwei Yin, Qiang Zhang, Richard Zhang, William T Freeman, Fredo Durand, Eli Shechtman, and Xun Huang. From slow bidirectional to fast autoregressive video diffusion models. In *CVPR*, 2025.

[27] Weili Nie, Julius Berner, Nanye Ma, Chao Liu, Saining Xie, and Arash Vahdat. Transition matching distillation for fast video generation. *arXiv preprint arXiv:2601.09881*, 2026.

[28] Shuai Yang, Wei Huang, Ruihang Chu, Yicheng Xiao, Yuyang Zhao, Xianbang Wang, Muyang Li, Enze Xie, Yingcong Chen, Yao Lu, et al. Longlive: Real-time interactive long video generation. *arXiv preprint arXiv:2509.22622*, 2025.

[29] Yang Jin, Zhicheng Sun, Ningyuan Li, Kun Xu, Hao Jiang, Nan Zhuang, Quzhe Huang, Yang Song, Yadong MU, and Zhouchen Lin. Pyramidal flow matching for efficient video generative modeling. In *ICLR*, 2025.

[30] Lvmin Zhang, Shengqu Cai, Muyang Li, Gordon Wetzstein, and Maneesh Agrawala. Frame context packing and drift prevention in next-frame-prediction video diffusion models. In *NeurIPS*, 2025.

[31] Kiwhan Song, Boyuan Chen, Max Simchowitz, Yilun Du, Russ Tedrake, and Vincent Sitzmann. History-guided video diffusion. In *ICML*, 2024.

[32] Yuwei Guo, Ceyuan Yang, Hao He, Yang Zhao, Meng Wei, Zhenheng Yang, Weilin Huang, and Dahua Lin. End-to-end training for autoregressive video diffusion via self-resampling. *arXiv preprint arXiv:2512.15702*, 2025.

[33] Kevin Lu and Thinking Machines Lab. On-policy distillation. *Thinking Machines Lab: Connectionism*, 2025. https://thinkingmachines.ai/blog/on-policy-distillation.

[34] Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.

[35] Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. On-policy context distillation for language models. *arXiv preprint arXiv:2602.12275*, 2026.

[36] Shanchuan Lin, Xin Xia, Yuxi Ren, Ceyuan Yang, Xuefeng Xiao, and Lu Jiang. Diffusion adversarial post-training for one-step video generation. In *ICML*, 2025.

[37] Shanchuan Lin, Ceyuan Yang, Hao He, Jianwen Jiang, Yuxi Ren, Xin Xia, Yang Zhao, Xuefeng Xiao, and Lu Jiang. Autoregressive adversarial post-training for real-time interactive video generation. In *NeurIPS*, 2025.

[38] Yuchao Gu, Weijia Mao, and Mike Zheng Shou. Long-context autoregressive video modeling with next-frame prediction. *arXiv preprint arXiv:2503.19325*, 2025.

[39] Hansi Teng, Hongyu Jia, Lei Sun, Lingzhi Li, Maolin Li, Mingqiu Tang, Shuai Han, Tianning Zhang, WQ Zhang, Weifeng Luo, et al. MAGI-1: Autoregressive video generation at scale. *arXiv preprint arXiv:2505.13211*, 2025.

[40] Guibin Chen, Dixuan Lin, Jiangping Yang, Chunze Lin, Junchen Zhu, Mingyuan Fan, Hao Zhang, Sheng Chen, Zheng Chen, Chengcheng Ma, et al. Skyreels-v2: Infinite-length film generative model. *arXiv preprint arXiv:2504.13074*, 2025.

[41] LightX2V Contributors. Lightx2v: Light video generation inference framework. https://github.com/ModelTC/lightx2v, 2025.

[42] FastVideo Team. Causalwan2.2-i2v-a14b-preview-diffusers. https://huggingface.co/FastVideo/CausalWan2.2-I2V-A14B-Preview-Diffusers, 2025.

[43] Erwann Millon. Krea realtime 14b: Real-time video generation, 2025.

[44] Haoyang Huang, Guoqing Ma, Nan Duan, Xing Chen, Changyi Wan, Ranchen Ming, Tianyu Wang, Bo Wang, Zhiying Lu, Aojie Li, Xianfang Zeng, Xinhao Zhang, Gang Yu, Yuhe Yin, Qiling Wu, Wen Sun, Kang An, Xin Han, Deshan Sun, Wei Ji, Bizhu Huang, Brian Li, Chenfei Wu, Guanzhe Huang, Huixin Xiong, Jiaxin He, Jianchang Wu, Jianlong Yuan, Jie Wu, Jiashuai Liu, Junjing Guo, Kaijun Tan, Liangyu Chen, Qiaohui Chen, Ran Sun, Shanshan Yuan, Shengming Yin, Sitong Liu, Wei Chen, Yaqi Dai, Yuchu Luo, Zheng Ge, Zhisheng Guan, Xiaoniu Song, Yu Zhou, Binxing Jiao, Jiansheng Chen, Jing Li, Shuchang Zhou, Xiangyu Zhang, Yi Xiu, Yibo Zhu, Heung-Yeung Shum, and Daxin Jiang. Step-video-ti2v technical report: A state-of-the-art text-driven image-to-video generation model, 2025.

[45] Patrick von Platen, Suraj Patil, Anton Lozhkov, Pedro Cuenca, Nathan Lambert, Kashif Rasul, Mishig Davaadorj, Dhruv Nair, Sayak Paul, William Berman, Yiyi Xu, Steven Liu, and Thomas Wolf. Diffusers: State-of-the-art diffusion models. https://github.com/huggingface/diffusers, 2022.

[46] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[47] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.

[48] Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, Yaohui Wang, Xinyuan Chen, Limin Wang, Dahua Lin, Yu Qiao, and Ziwei Liu. VBench: Comprehensive benchmark suite for video generative models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

[49] Ziqi Huang, Fan Zhang, Xiaojie Xu, Yinan He, Jiashuo Yu, Ziyue Dong, Qianli Ma, Nattapol Chanpaisit, Chenyang Si, Yuming Jiang, et al. Vbench++: Comprehensive and versatile benchmark suite for video generative models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.